IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2024

DIGITAL ELECTRONICS & COMPUTER ARCHITECTURE

Tuesday, 7 May 2:30 pm

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions. Question One carries 40 marks. Question Two and Question Three carry 30 marks each.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : G.A. Constantinides, T. Clarke
Second Marker(s) : T. Clarke, G.A. Constantinides

© Imperial College London

DIGITAL ELECTRONICS AND COMPUTER ARCHITECTURE

1. a) Simplify the following Boolean expression. You may use any method, but the method must be clear from your answer.

$$\overline{A}\,\overline{B}+A\left(\overline{B}+\overline{C}\right)+C$$

[4]

b) Simplify the Boolean expression $Y=\prod\{0,1,4,5,6\}$ using a Karnaugh map, leaving your answer in product-of-sums form. Show all your working.

[4]

c) Rewrite the Boolean expression $\overline{A+\overline{B}}\ \left(\overline{C}+\overline{D}\right)$ in minimal sum-of-products form. Show all your working.

[4]

d) Convert the 8-bit values 01001111 and 11001111 to decimal, (i) first interpreting them each as an unsigned integer (ii) then interpreting them each as a two's complement integer.

[4]

e) From Figure 1.1, (i) write down a Boolean expression for X^+ (i.e. the value of X in the next clock cycle) in terms of A and B, and (ii) hence or otherwise simplify the circuit to use fewer flip-flops.

[4]

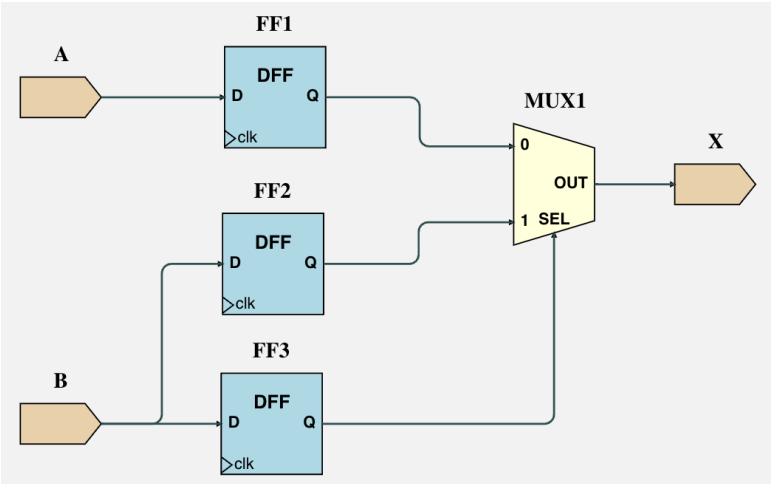
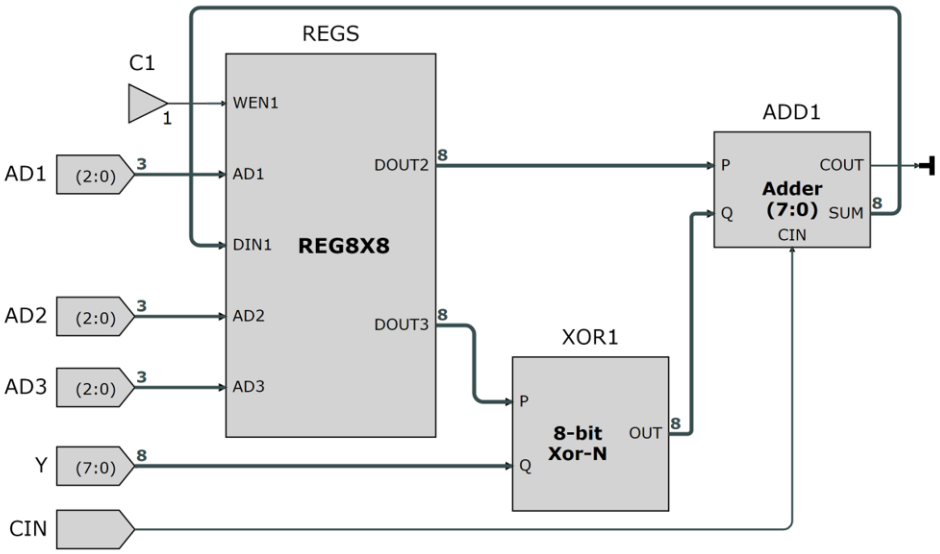


Figure 1.1 A sequential circuit.



(a)

Operation	AD1(2:0)	AD2(2:0)	AD3(2:0)	Y(7:0)	CIN
`R3 := R4 + R5`					
`R2 := R0 - R1`					
`R6 := 2 * R6`					

(b)

Figure 1.2 A CPU datapath

f) Figure 1.2a shows a CPU datapath comprising a register file, 8 bit adder, and 8 XOR gates. Copy the table in Figure 1.2b, adding the values of the signals in each row required to implement the specified operation.

[9]

g) Figure 1.3 shows an EEP1 assembly program loaded into instruction memory starting from address 0. The EEP1 CPU executes this program. Calculate, in hexadecimal, the final values of the CPU registers changed by the program, and the addresses and new values of memory locations written by the program. Write clearly in a table the final values of each register, memory address, and memory data value.

[11]

```
MOV R0, #5
MOV R1, #7
SUB R2, #30
ADD R0, #-1
ADC R1, #0
STR R0, [#6]
STR R1, [#255]
MOV R0, #250
EXT 255
MOV R1, #1
MOV R3, #-1
SBC R3, #0
MOV R4, #15
MOV R5, #15
LSL R4, R4, #14
XSR R5, R5, #1
JSR 100
```

Figure 1.3 EEP1 assembly program

2. Recall that an n-bit sign-and-magnitude integer representation $X(n-1:0)$ uses a single bit, $X(n-1)$, to indicate the sign of the number and the remaining bits $X(n-2:0)$ to indicate its magnitude. For a number x, the sign bit is zero for $x > 0$ and 1 for $x < 0$. The magnitude is represented as an unsigned binary integer.

a) Write down an expression in terms of n for the range of possible values representable by an n-bit sign-and-magnitude integer.

[2]

b) Design a circuit to check whether a 4-bit sign-and-magnitude integer represents zero. The output of your circuit should be 1 if and only if the number represented is zero. You may draw your circuit using only AND, OR and/or NOT gates. Any AND and OR gates can have an arbitrary number of inputs.

[2]

c) Draw a Moore model state diagram of a FSM which has no inputs (except for the clock), and a 4-bit sign-and-magnitude output, and counts in the sequence $0, 1, -1, 2, -2, 4, -4$, repeating this sequence indefinitely.

[7]

Parts (d) and (e) of this question explore two different implementations of this FSM.

d) Using the sign-and-magnitude representation of the output value as the assigned state of the FSM, write out Karnaugh maps for each of the next-state bits in terms of the current state. You may assume that the FSM will enter a well-defined state at power on, so you may exploit any unused combinations of state bits as don't cares. Identify appropriate groups corresponding to minimal SOP implementations of each bit. You do not have to write the Boolean expressions or draw the circuit implementation.

[8]

e) An alternative implementation of the FSM uses a smaller number of state bits than output bits. In this case we use the unsigned binary representation of the state $0, 1, 2, 3, 4, 5, 6$ to correspond to the count sequence $0, 1, -1, 2, -2, 4, -4$. For this choice of assigned state, write out Karnaugh maps for each of the next-state and output bits in terms of the current state. You may assume that the FSM will enter a well-defined state at power on, so you may exploit any unused combinations of state bits as don't cares. Identify appropriate groups corresponding to minimal SOP implementations of each bit. You do not have to write the Boolean expressions or draw the circuit implementation.

[11]

3. Figure 3.1 shows an EEP1 assembly code program that has inputs $R0$, $R1$ it does not modify. The program uses a loop to implement an unsigned 16 bit multiply operation with a truncated 16 bit product:

$$R2 := (u(R0) * u(R1))(15:0)$$

a) Assuming that the loop executes its body at least once, write a C++ `while` loop which is equivalent to Figure 3.1 replacing the shift instructions by the equivalent arithmetic. You may find it useful to know that in C++ the type `unsigned short` is a 16 bit unsigned integer with `/` an unsigned integer division operation (rounding down). `(x & 1)` is non-zero only if the least significant bit of `x` is 1.

[6]

b) Determine the conditions on $R0, R1$ for the `while` loop never to execute its body and hence or otherwise show that its result is also the same in that case as the assembly code.

[2]

c) Write expressions for $u(X)$ and $z(X)$, where $X = X(15:0)$, in terms of $X_S = X(15)$ and $X' = u(X(14:0))$. Hence, or otherwise, show that:

$$(z(R0) * z(R1))(15:0) = (u(R0) * u(R1))(15:0)$$

Explain why, despite this, $R3$ in Figure 3.1 cannot be treated as a signed number.

[4]

d) Add EEP1 assembly code for the entry and exit of the subroutine to the code in Figure 3.1 to make a subroutine `MULTIPLY` complying with the EEP1 Subroutine Call Protocol. Write in EEP1 assembly code a program `SUMOFPRODUCTS` that implements $R0 := R0 * R1 + R2 * R2$ by calling your subroutine twice. Credit will be given for correct answers with the minimum number of instructions.

[8]

e) Write a pointer diagram showing where the registers used by `SUMOFPRODUCTS` are stored on the stack relative to the stack pointer when execution of `SUMOFPRODUCTS` has finished the subroutine entry code in `MULTIPLY`.

[4]

f) Assume that at entry into the code in Figure 3.1 $R0 \neq 0$ and m, k are defined from the bits of $R0$ as shown in Figure 3.2. Determine how many times the loop is executed. Hence, or otherwise, find the code execution time in clock cycles when executed on the EEP1 CPU. Calculate the average instruction throughput on a pipelined CPU similar to EEP1 with a clock frequency of 100MHz and two pipeline stall cycles every time a jump is taken.

[6]

```
START  MOV R2, #0
       MOV R3, R0
       MOV R4, R1
LOOP   LSR R3, R3, #1
       JCC SKIP
       ADD R2, R2, R4
SKIP   LSL R4, R4, #1
       CMP R3, #0
       JNE LOOP
```

Figure 3.1 EEP1 Multiply Program

m : The largest i for which $R0(i) = 1$
k : The number of bits in R0 equal to 1

Figure 3.2 Definition of n and k

DECA Spring Term Exam Notes 2024

(These notes will be distributed in the Exam printed greyscale 2 slides per side)

- The EEP1 CPU is a 16 bit Harvard architecture, with 16 bit addresses and word addressing for both data and code, hence 2^{17} bytes of instruction memory and 2^{17} bytes of data memory.
- At start all EEP1 registers and flags are zero
- All EEP1 instructions take 1 clock cycle to execute

EEP1 Machine Code Overview

SHIFTOPC(1:0)	Shift
0	LSL
1	LSR
2	ASR
3	XSR

ALUOPC	INS(8)=0	INS(8)=1
0	MOV[1] Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP[1] Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

[1]Instruction does not use Rc, this field is (0)

legend	Meaning
Imm4	4 bit unsigned immediate
Imms5	5 bit signed immediate
Imms8	8 bit signed immediate
(0)	Must be 0 for current instructions, non-zero values are reserved for expansion

	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EEP1 machine code	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							
	Load / Store	1	0	MOPC	(0)	a			0	b			Imms5 (OFFSET)				
									1	Imms8 (IMM)							
	Jump	1	1	0	0	JMPOPC			Imms8 (JOFFSET)								
	EXT	1	1	0	1	(0)			Imms8 (IMM)								
	Not used	1	1	1	X	X											

EEP1 ALU Instructions

ALUOPC	INS(8)=0	INS(8)=1
0	MOV[1] Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP[1] Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

[1]Instruction does not use Rc, this field is (0)

SHIFTOPC(1:0)	Shift
0	LSL
1	LSR
2	ASR
3	XSR

Legend	Meaning
Imm4	4 bit unsigned immediate
Imms8	8 bit signed immediate
(0)	Must be 0 for current instructions, non-zero values are reserved for expansion

EEP1 machine code	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							

EEP1 ALU instructions: detail

OPCALU	Mnemonic	ALU.Out: INS(8)=0	ALU.Out: INS(8) = 1	Write FlagV & FlagC[1]	Write register	
0	MOV	Rb	Imm	No	Ra	move
1	ADD	Ra + Rb	Ra + Imm	Yes	Rc (Ra if INS(8)=1)	arithmetic
2	SUB	Ra - Rb	Ra – Imm	Yes	Rc (Ra if INS(8)=1)	
3	ADC	Ra + Rb + C	Ra + Imm + C	Yes	Rc (Ra if INS(8)=1)	
4	SBC	Ra – Rb + (C-1)	Ra – Imm + (C-1)	Yes	Rc (Ra if INS(8)=1)	
5	AND	Ra & Rb	Ra & Imm	No	Rc (Ra if INS(8)=1)	"bitwise" logical
6	CMP	Ra - Rb	Ra - Imm	Yes	no	comparison
7	LSR, ASR, XSR, LSL	depends on SHIFTOPC		Yes[2]	Ra	shift

Flag	
FlagN	z(ALU.Out) < 0
FlagZ	ALU.Out = 0
FlagC	carry
FlagV	signed overflow

&	16 bit C++ bitwise logical AND operator

FlagN & **FlagZ** are written by **all** ALU instructions

Flags are only written by ALU instructions

[1]FlagC
- 1 bit flip-flop in CPU used to store carry for multi-word addition and shifts
- Holds previous value unless written

C = FlagC.Q

[2]Shift instructions write FlagC with *last bit shifted out*, FlagV undefined.

EEP1 Memory Instructions

LDR	**Load Register**
STR	**Store Register**

INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	0	MOPC	(0)	a			0	b			Imms5 (OFFSET)				
								1	Imms8 (IMM)							

Field	Limits	Allowed Syntax
Imms5	-16 .. +15	`#0xA, #12, #-0x10, #-15`
Imms8	-128 .. 127	`#0x34, #100, #-33`
a, b	0..7	`R0, R5, R7`

Type	MOPC	I(8)	Assembly Syntax	Examples	Effective Address	Operation
register offset load	0	0	`LDR Ra, [Rb, #Imms5]`[3] `LDR Ra, [Rb]`[2]	`LDR R4,[R6,#-12]` `LDR R2, [R7]`	Rb+Imms5[1] Rb	Ra := mem[Rb + Imms5[1]] Ra := mem[Rb]
direct load	0	1	`LDR Ra, [#Imms8]`[4]	`LDR R0,[#220]`	Imms8[1]	Ra := mem[Imms8[1]]
register offset store	1	0	`STR Ra, [Rb, #Imms5]`[3]	`STR R6, [R6, #9]`	Rb+Imms5[1]	mem[Rb + Imms5[1]] := Ra
direct store	1	1	`STR Ra, [#Imms8]`[4]	`STR R0, [R1, #0x66]`	Imms8[1]	mem[Imms8[1]] := Ra

[1] Imms5 is a 5 bit 2's complement signed number sign extended to 16 bits, Imms8 is 8 bit signed extended to 16 bits.
[2] Special case: assembly syntax allows Imms5 offset to be omitted in which case it has value 0 in the machine code.
[3] Register offset addressing. [4] Direct addressing.

EEP1 Jump instructions

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							

JMPOPC(3:1)	NZCV condition	Condition on data for jump with JMPOPC(0)=0	JMPOPC(0) = 0	JMPOPC(0) = 1[3]
0	1	always	**JMP** (Always jump)	**NOOP** (Never jump)
1	Z	result = 0	**JEQ** (Equal)	**JNE** (Not Equal)
2	C	C=1 ≡ u(Ra) ≥ u(Op)[1]	**JCS** (Carry Set / Unsigned Higher or Equal)	**JCC** (Carry Clear / Unsigned Lower)
3	N	z(result) < 0	**JMI** (Minus)	**JPL** (Plus)
4	$\bar{N} \oplus V$	z(Ra) ≥ z(Op)[1]	**JGE** (Signed Greater or Equal)	**JLT** (Signed less than)
5	$(\bar{N} \oplus V).\bar{Z}$	z(Ra) > z(Op)[1]	**JGT** (Signed Greater Than)	**JLE** (Signed Less than or Equal)
6	$C.\bar{Z}$	u(Ra) > u(Op)[1]	**JHI** (Unsigned Higher)	**JLS** (Unsigned Lower or Same)
7	1	always	**JSR** *(R7[4] := PC + 1, PC := PC + JOFFSET)*	**RET** *(PC := R7)*

[1] **JGE,JGT,JHI, JCS** jump conditions jump based on the flag values. The table column 3 shows the comparison of **Ra** and **Rb** after a previous `SUB Ra, Rb` instruction which delivers those NZCV values, or the condition on an ALU result which delivers the required NZ values.

[2] **JSR** and **RET** always jump and are used to *jump to* and *return from* subroutine see notes. **RET** does not use **Joffset**.

[3] **JMPOPC(0) = 1** inverts all conditions, except for JSR/RET

[4] Hardware is simpler because **JSR**/**RET** write/read R7 since **Ra** is normally read and written and for these instructions **a** = **JMPOPC(3:1)** = 7

Jump Instructions	Assembly	Operation	Examples
All except RET, JSR	JEQ JOFFSET; JEQ SYMBOL	PC := PC + JOFFSET if condition is true	`JMP -1 ; JGT LOOP ; JSR SUB1`
RET	RET	PC := R7	`RET`
JSR	JSR JOFFSET	R7 := PC+1; PC := PC + JOFFSET	`JSR 10`

EXT instruction

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	1	(0)				Imm8 (IMM)							

Assembly example

Memory address

0x0100 EXT 0x23
0x0101 JMP 0xF0 ;modified jump instruction

jump

0x24F1 …

0x2300 *EXT IMM8 = 0x23*
0x00F0 *JMP IMMS8 = 0xF0*
0x0101 *Address (PC) of JMP instruction*
0x24F1 *Jump target address*

- EXT modifies the instruction immediately after it, replacing bits (15:8) of Imm16 (otherwise from Imms8 sign extension) with IMM rom the EXT instruction
- NB this means the 8 Imms8 bits from the modified instruction are not sign extended because the extended part is replaced.
- Applies to:
  - Jump instructions,
  - **Direct** memory instructions (not register offset)
  - ALU instructions with immediate operand

7

7

EEP1 Subroutine Call Protocol determines how registers are used by subroutines

- The EEP1 subroutine stack is a *downwards growing, empty stack pointer* stack using R6 as stack pointer.
- The protocol fixes the usage of each register for all subroutines and determines what must be saved or restored on subroutine entry/exit.
  - **Scratch** registers (can be changed by any subroutine, not saved/restored at entry/exit)
  - **Persistent** registers (must be saved/restored at entry exit by callee subroutine on callee's stack frame if used)
  - **Stack Pointer**
  - **Return Address**
- All ISAs use protocols similar to this so that compiled code can be turned into subroutines
- Scratch registers are also used if needed for 1st three subroutine inputs and outputs

EEP1 standard subroutine call protocol (the *caller* calls the *callee*)			
Register	Content Type	Saved by callee on subroutine entry/exit if used	Persistent across subroutine calls, can be used for storage by caller
R7	**Return Address**	**yes**	**no**
R6	**Stack Pointer (SP)**	**n/a**	**yes, no***
R5	persistent	yes	yes
R4	persistent	yes	yes
R3	persistent	yes	yes
R2	scratch (or input 3, or output)	no	no
R1	scratch (or input 2, or output)	no	no
R0	scratch (or input 1, or output)	no	no

*R6 can't be used for storage by caller, because it must always contain the stack pointer

8

8

Notation for DECA Spring 2024 (1)

- **High, low.** Synonym for digital logic values 1, 0.
- **X.** *don't care* value that does not affect operation
- **Clocked logic.** Logic with edge triggered flip-flops. All clocks are assumed connected together to a single global clock.
- **Combinational logic.** Logic with gates and no flip-flops, also no *latches* AKA RS flip-flops.
  - NB we will not use *latches* in this module, therefore combinational logic loops (that might make latches) are also not allowed. Every such loop must be broken by a D -> Q flip-flop path, or equivalent memory path.
- **Asynchronous (synchronous) RAM.**
  - The *output* is valid in the same cycle (the next cycle) as the *address*. In both cases the write operation is synchronous, and locations change to a newly written value in the cycle after that in which write is 1 and address, din specify the written location and value.
- **Register EN** – enable input, if 1 then something is written
  - Named WEN on RAMs and register files
- **Decoder EN** – enable input, if 0 then all outputs are 0
  - Note differing use of EN on clocked and combinational logic.
- **A := B.** Notation to set register A equal to B on active clock edge at end of current clock cycle. B can be a register or a number.
- **Mem[a]** – memory *location* with *word* address a. **Mem[a] := B** – set memory location address a to contents of B (or B – if B is a number).
- **Mem_{32}[a], Mem_8[a]** word and byte locations in a 32 bit system with byte addressing such as ARM. Word or byte memory read and write using **:=** as above for Mem[a].
- **MUX.SEL** – signal SEL on block MUX
- **MUX.SEL** input – selects which of MUX data inputs passes to output. Data inputs are numbered according to their SEL value.
- **IR(7)** – *selected bit 7* of binary data or register IR. Bits of n bit bus are numbered *little-endian* with bit 0 being LSB, bit n-1 MSB.
- **IR(3:0)** – *slice* of binary data or register IR. Bits from 3 to 0 inclusive.

Notation for DECA Spring 2024 (2)

- **Byte.** 8 bits of data. Possibly interpreted as unsigned (0..255) or signed (-128..127) number.
- **Word.** Binary data in one CPU memory location or CPU data register. Typical values 8, 16, 32, 64 bits. Sometimes interpreted as unsigned or signed number.
- **MSW** (**LSW**) Most (Least) Significant Word, when words are joined to make a binary number with more bits
- **MSB (LSB). MS byte, LS byte**. The most significant or least significant bit or byte respectively of a binary word.
- **R1:R2**. The binary value made from the bits of R1 (MSBs) and R2 (LSBs). If R1, R2 are 16 bit: (R1:R2)(31:16) = R1 and (R1:R2)(15:0) = R2
- $\mathbf{A_N}$ - Value of the **A** signal in cycle N. NOT **A(N)** the Nth bit of A!
- **0xA30, 0b010**, hex, binary representations of numbers.
- **R0, Rn** CPU register 0, n (e.g. n = 0..3 or n = 0...15). CPU registers have numbers R0, R1, ... For a register file with named registers REGn REG[i] indicates register REGi.
- **u(A), z(A),** Respectively the *unsigned* and *signed* numeric interpretation of signal A. A must have a known width *n*, in which case u(A) and z(A) differ only when the sign bit A(n-1) is 1. *Bit slicing* is allowed on integers derived from u(), z(): (u(P(n-1:0))+u(Q(n-1:0)))(n-1:0) is the n Sum bits of a binary adder with n-bit inputs P, Q. The carry output bit from this could be written as: (u(P(n-1:0))+u(Q(n-1:0)))(n)
- $\mathbf{A_S}$ = A(n-1), the most significant (sign) bit of A(n-1 : 0).
- **A'** If A= A(n-1 : 0) has n bits, **A'** = A(n-2 : 0). If used in a numeric context this is interpreted as u(A') an unsigned integer.
- m **mod** n. The same as `m % n` in C++. m **mod** 2^n is the unsigned value of the LS 8 bits of m. If m is negative this is two-'s complement: -1 `mod` 2^8 = 255; 260 `mod` 2^8 = 4; x `mod` 2^n is x truncated to its n least significant bits. Division is assumed integer unless otherwise stated.